

第9讲 神经网络的训练技巧

神经网络与深度学习 2026

信息科学技术学院 吴瀚霖

hlwu@bfsu.edu.cn

Math Warning !

机器学习 $\approx$ 找函数



1

我有什么选择

2

什么样的 f 更好

3

选一个最好的

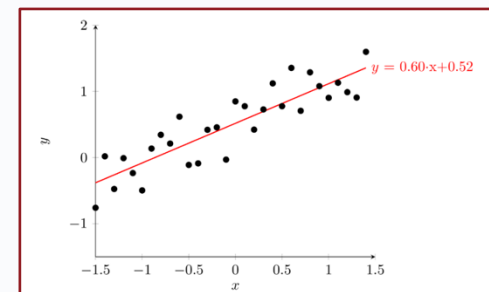
训练

模型：我有什么选择

线性模型

$$f(\mathbf{x}; \mathbf{w}, b) = \mathbf{w}^\top \mathbf{x} + b$$

如何确定 $\mathbf{w}$ 和 b ?



非线性模型

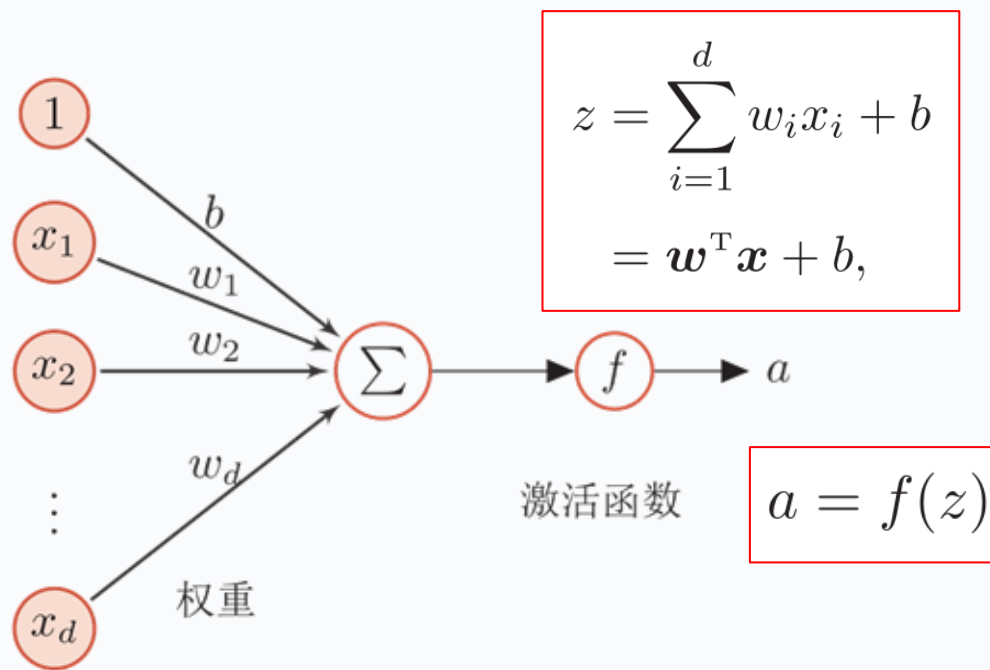
$$f(\mathbf{x}; \theta) = \phi_\theta(\mathbf{x})$$

ϕ 可以由神经网络表达的函数，参数为 θ

函数集合 $\mathcal{F}$ 也被称为假设空间 (Hypothesis Space)

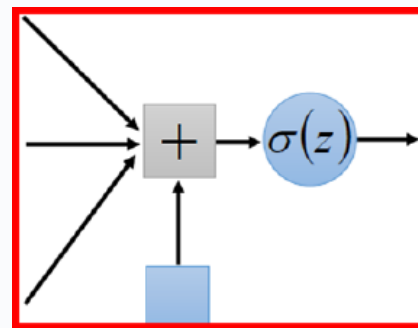
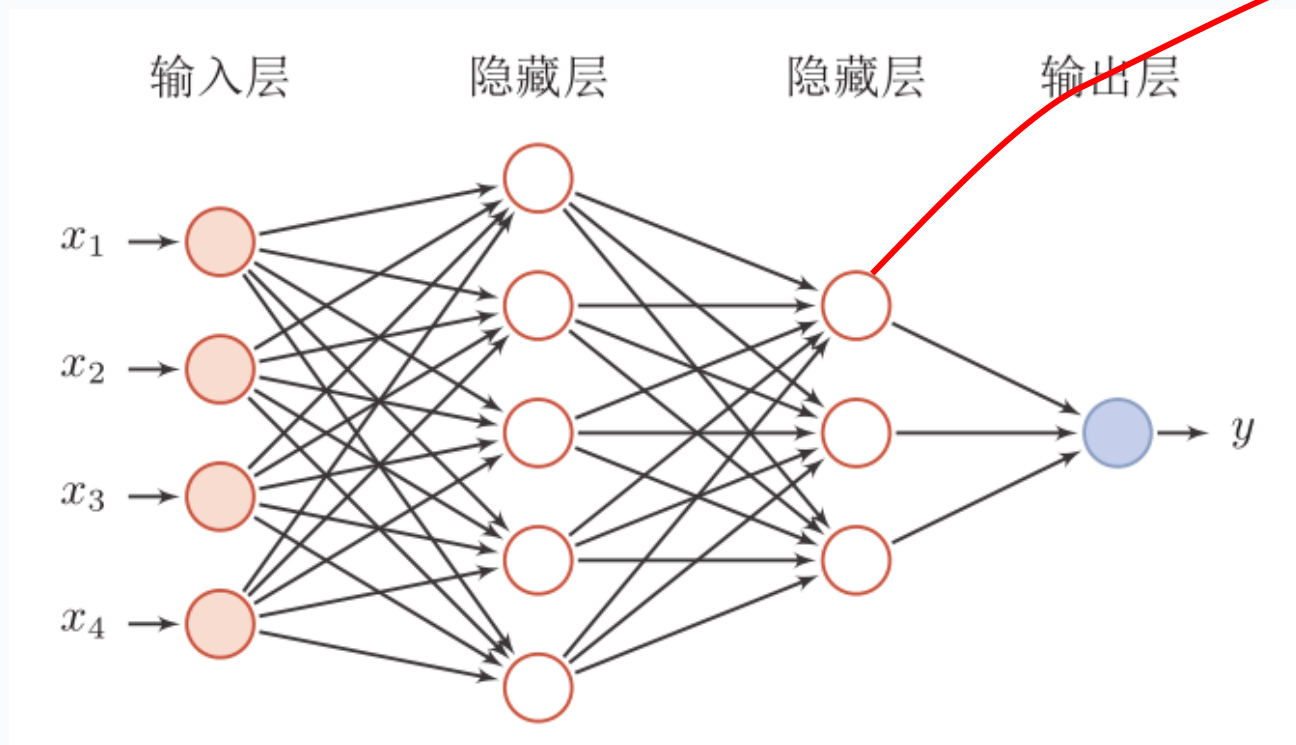
神经网络

神经元：最基本的组成单元



前馈神经网络

前馈神经网络



神经元

学习准则：什么样的 f 更好

损失函数 (Loss Function)

- 量化模型预测和标签之间的差异。
- 平方损失函数：多用于回归问题

$$\mathcal{L}(y, f(\mathbf{x}; \theta)) = \frac{1}{2} (y - f(\mathbf{x}; \theta))^2$$

- 交叉熵损失函数：多用于分类问题

$$\mathcal{L}(\mathbf{y}, f(\mathbf{x}; \theta)) = - \sum_{c=1}^C y_c \log f_c(\mathbf{x}; \theta)$$

优化目标 (经验风险)

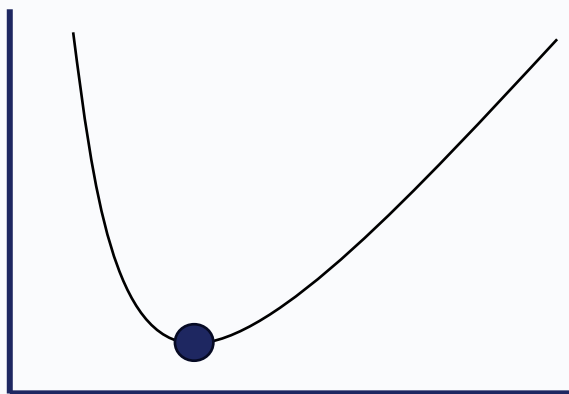
$$L(\theta) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}(y^{(n)}, f(x^{(n)}, \theta))$$

优化算法：选一个最好的

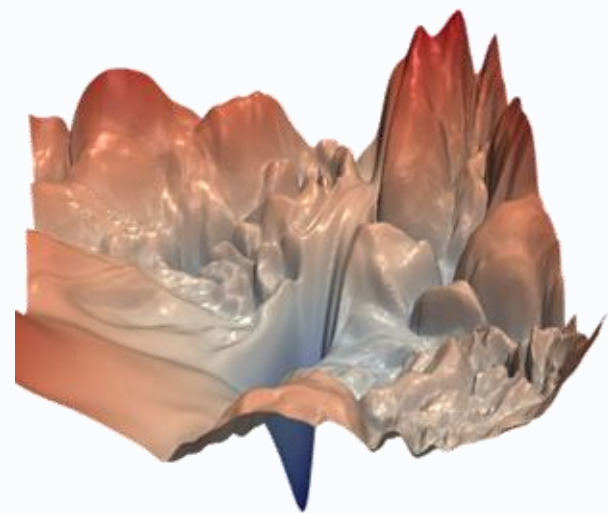
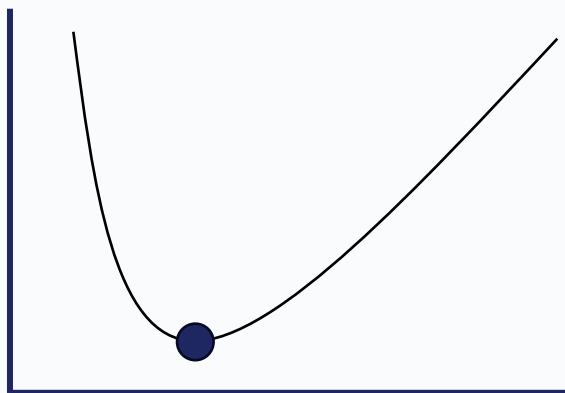
在确定了**假设空间 $\mathcal{F}$** 以及**学习准则**后

如何找到最优的模型 $f(x, \theta^*)$ 就成了一个最优化（Optimization）问题。

凸函数

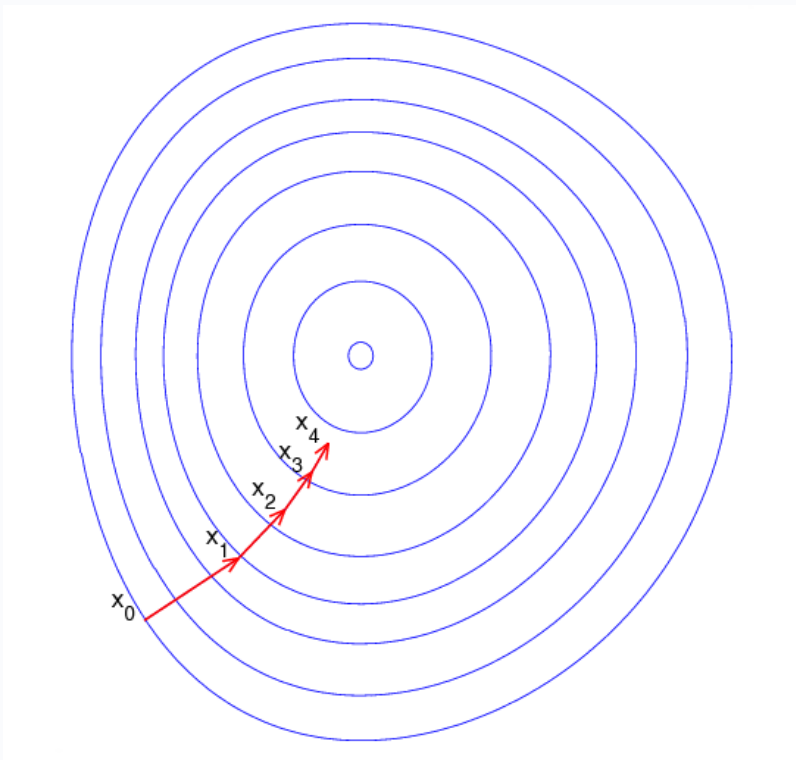


非凸函数



优化算法：选一个最好的

梯度下降算法



- 每一步都朝着目标靠近一点

$$\theta_{t+1} = \theta_t - \boxed{\alpha} \frac{\partial L(\theta)}{\partial \theta} \Big|_{\theta=\theta_t}$$

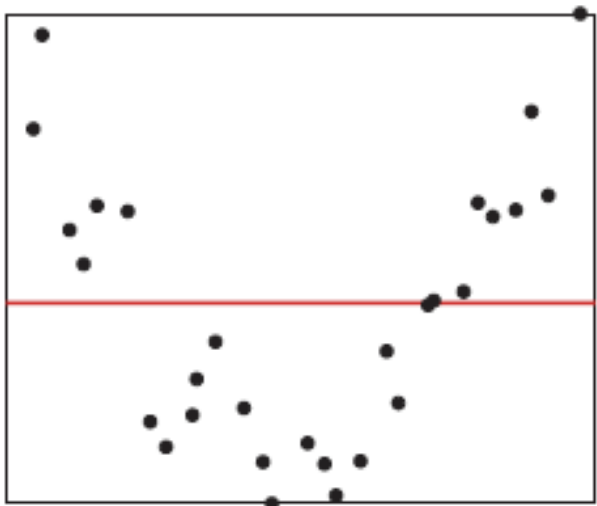
搜索步长 α 为学习率 (Learning Rate)

机器学习 $\neq$ 优化

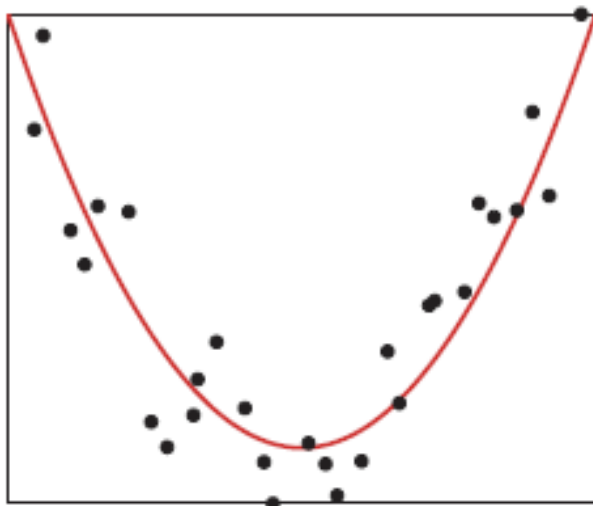
NO!

机器学习 = 优化?

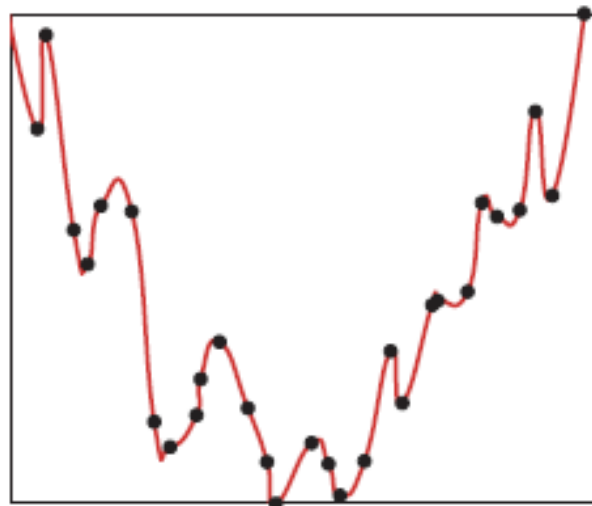
欠拟合



正常



过拟合



过拟合与欠拟合

NO!

机器学习 = 优化?

训练集上的误差

测试集上的误差

	低	高
低	YES!	? ? Bug
高	过拟合	欠拟合

介绍训练神经网络时常用的各种技巧

谈起一个训练神经网络有关的方法时，可以这样问自己

方法名称	改了哪个步骤	带来什么好处
.....		

1

我有什么选择

2

什么样的 f 更好

3

选一个最好的

介绍训练神经网络时常用的各种技巧

谈起一个训练神经网络有关的方法时，可以这样问自己

方法名称	改了哪个步骤	带来什么好处
.....		

更好的优化？

(训练集上)

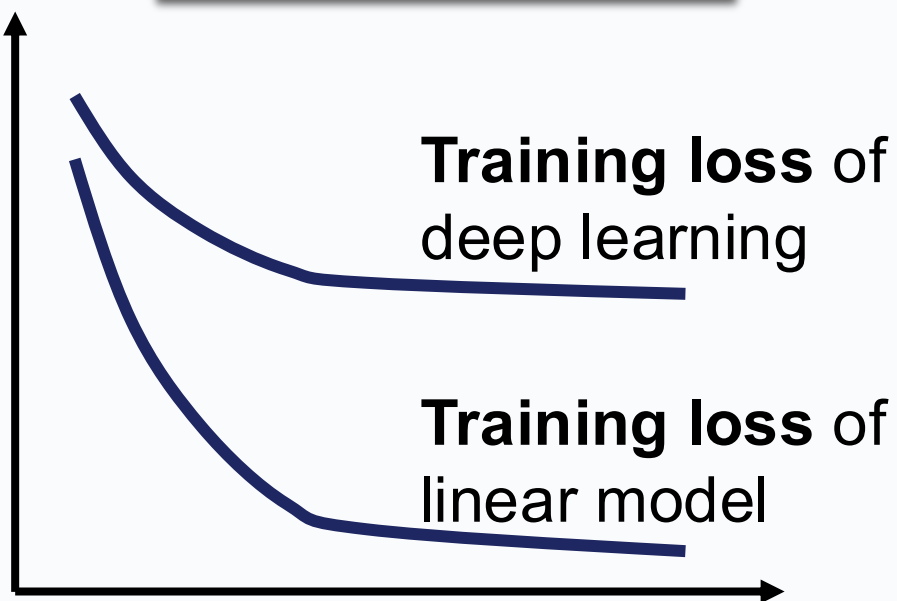
OR

更好的泛化性能？

(测试集上)

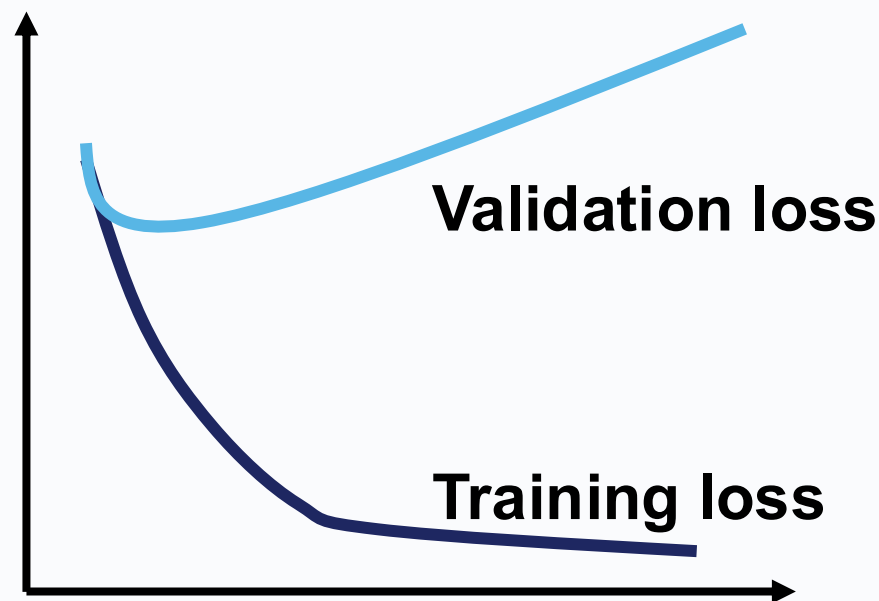
选择合适的技巧

优化出了问题 🤖



更好的优化

Overfitting



更好的泛化性能

寻找函数的三个步骤

1

我有什么选择

2

什么样的 f 更好

3

选一个最好的

更好的优化方法

Vanilla 梯度下降

$$\theta^* = \arg \min_{\theta} L(\theta)$$

➤ (随机地) 选取初始值 θ^0

➤ 计算梯度 $g^0 = \nabla L(\theta^0)$

$$\theta^1 \leftarrow \theta^0 - \eta g^0 \quad \leftarrow \quad 1 \text{ 次更新}$$

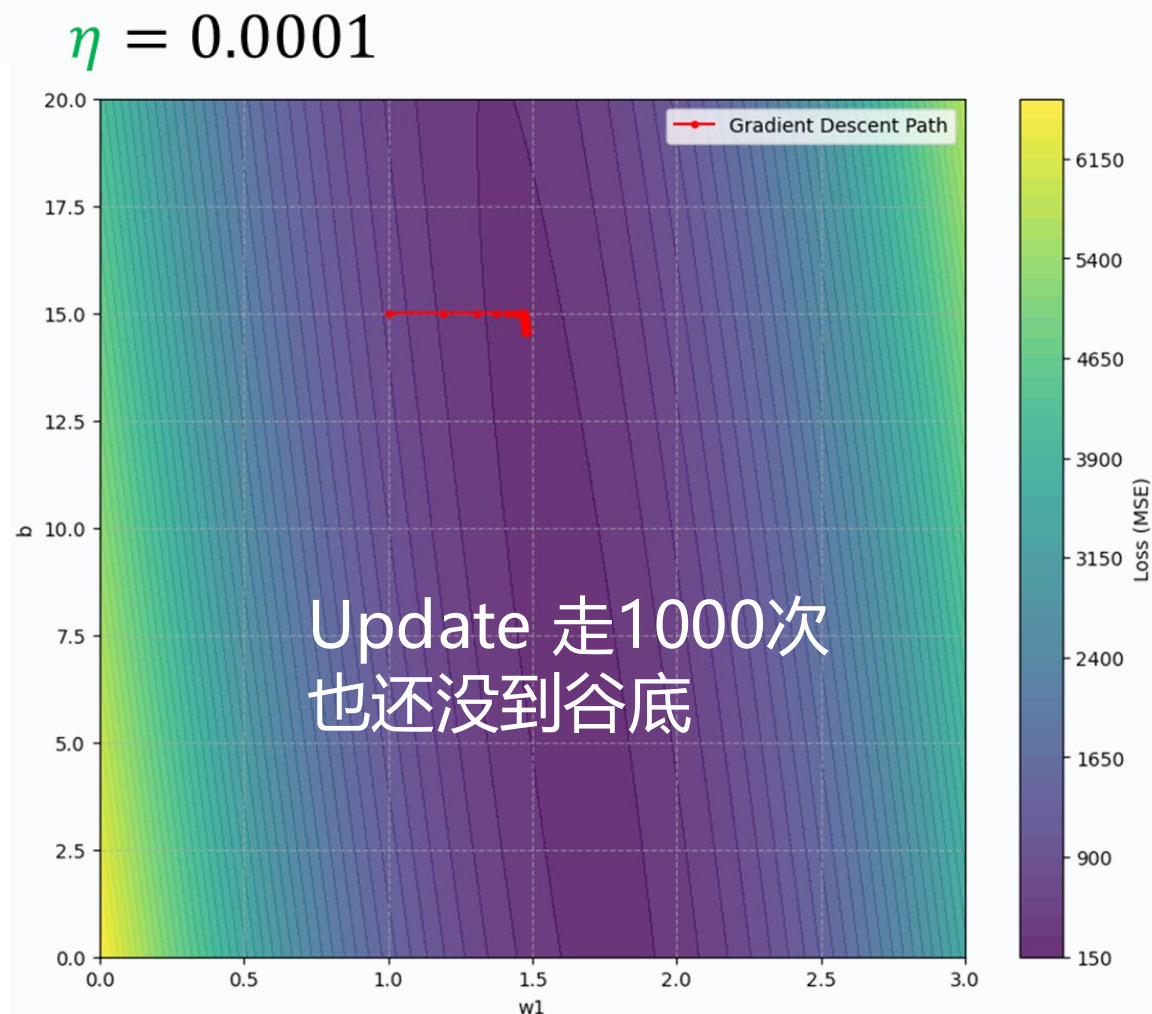
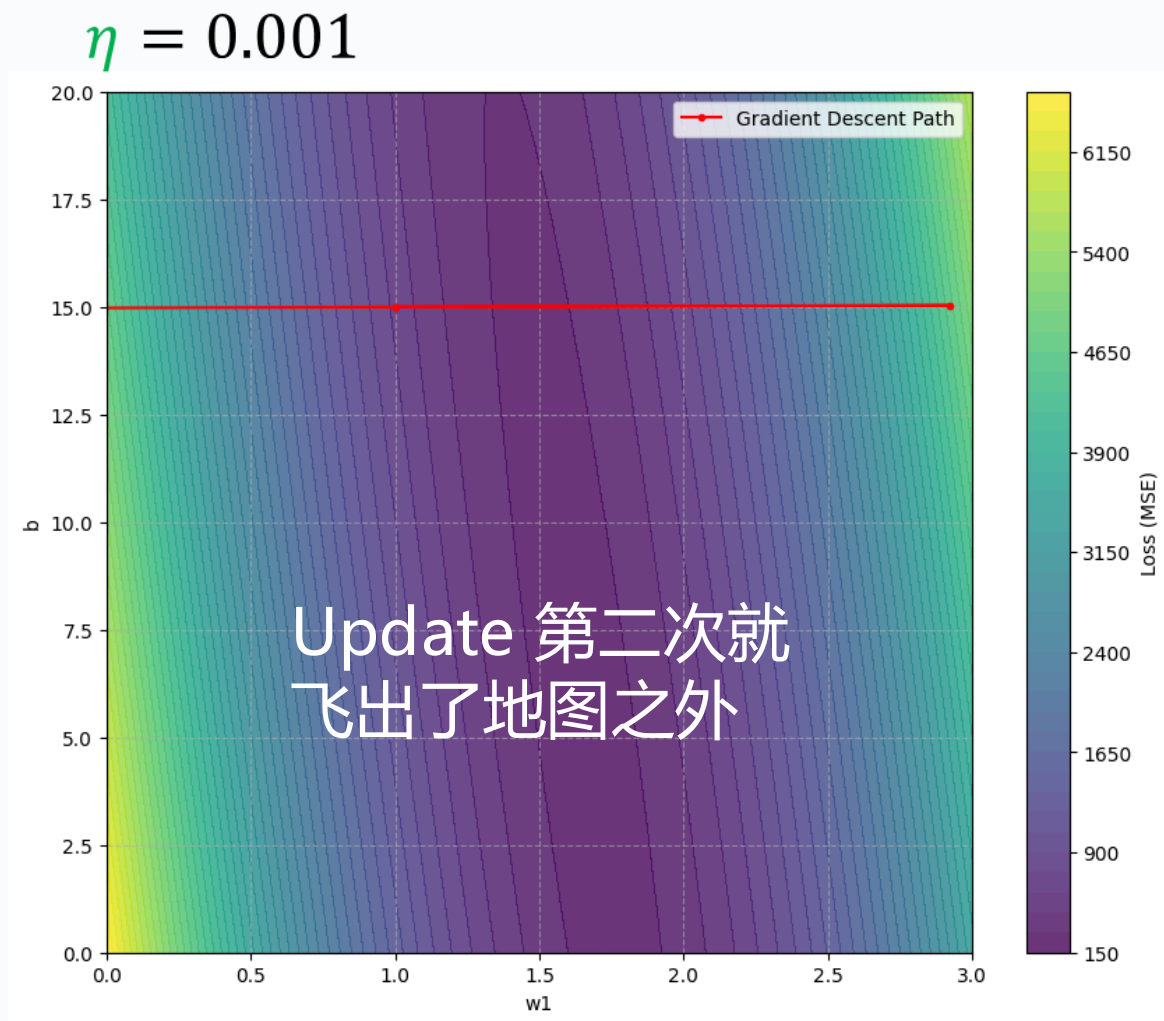
➤ 计算梯度 $g^1 = \nabla L(\theta^1)$

$$\theta^2 \leftarrow \theta^1 - \eta g^1$$

➤ 计算梯度 $g^2 = \nabla L(\theta^2)$

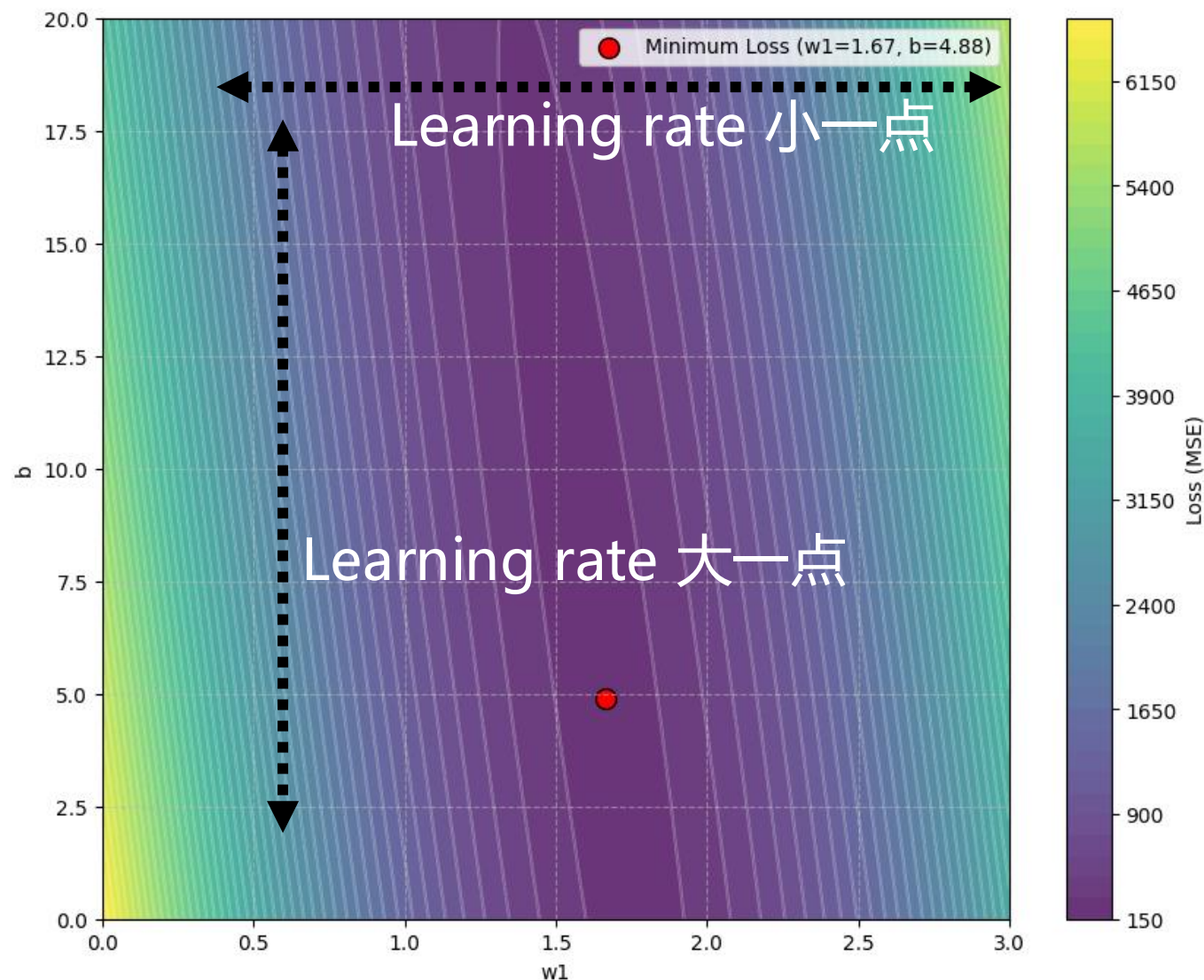
$$\theta^3 \leftarrow \theta^2 - \eta g^2$$

Learning Rate 不太好调

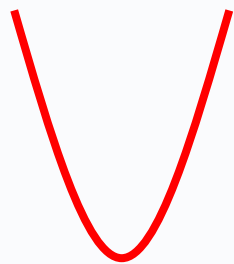


不同参数需要的Learning Rate可能不同

怎么知道哪个方向的变化率大、哪个小？



用历史经验决定 Learning Rate



Learning rate 小一点

$$g^0 = \begin{bmatrix} 500 \\ 0.4 \end{bmatrix}$$

$$g^1 = \begin{bmatrix} 432 \\ -0.3 \end{bmatrix}$$

$$g^2 = \begin{bmatrix} -211 \\ 0.2 \end{bmatrix}$$

$$g^3 = \begin{bmatrix} 139 \\ 0.1 \end{bmatrix}$$



Learning rate 大一点

优化算法：自适应学习率

$$\theta_i^{t+1} \leftarrow \theta_i^t - \frac{\eta}{\sigma_i^t} g_i^t$$

Adagrad

$$\theta_i^1 \leftarrow \theta_i^0 - \frac{\eta}{\sigma_i^0} g_i^0$$

$$\sigma_i^0 = \sqrt{(g_i^0)^2} = |g_i^0|$$

$$\theta_i^2 \leftarrow \theta_i^1 - \frac{\eta}{\sigma_i^1} g_i^1$$

$$\sigma_i^1 = \sqrt{\frac{1}{2} [(g_i^0)^2 + (g_i^1)^2]}$$

$$\theta_i^3 \leftarrow \theta_i^2 - \frac{\eta}{\sigma_i^2} g_i^2$$

$$\sigma_i^2 = \sqrt{\frac{1}{3} [(g_i^0)^2 + (g_i^1)^2 + (g_i^2)^2]}$$

⋮

$$\theta_i^{t+1} \leftarrow \theta_i^t - \frac{\eta}{\sigma_i^t} g_i^t$$

$$\sigma_i^t = \sqrt{\frac{1}{t+1} \sum_{i=0}^t (g_i^t)^2}$$

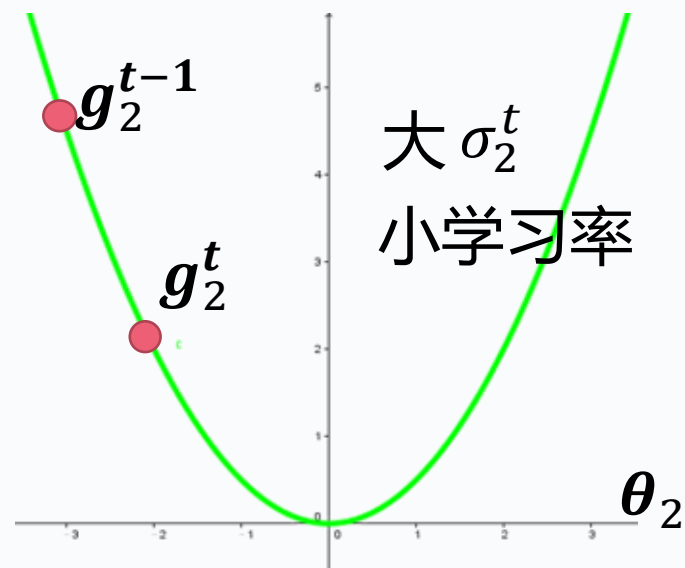
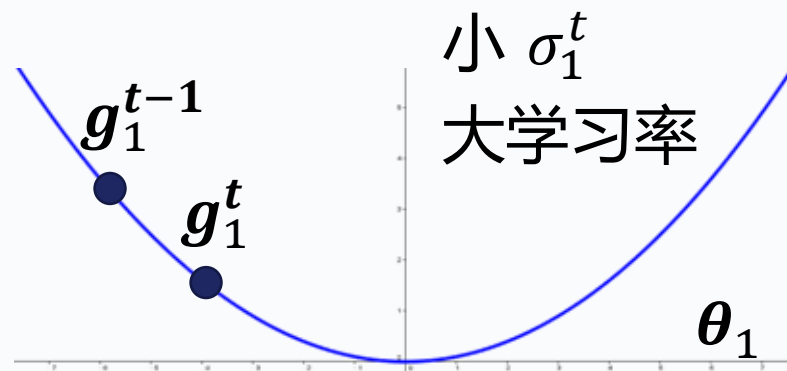
平均还是求和？

优化算法：自适应学习率

Adagrad

$$\theta_i^{t+1} \leftarrow \theta_i^t - \boxed{\frac{\eta}{\sigma_i^t}} g_i^t$$

$$\sigma_i^t = \sqrt{\frac{1}{t+1} \sum_{i=0}^t (g_i^t)^2}$$

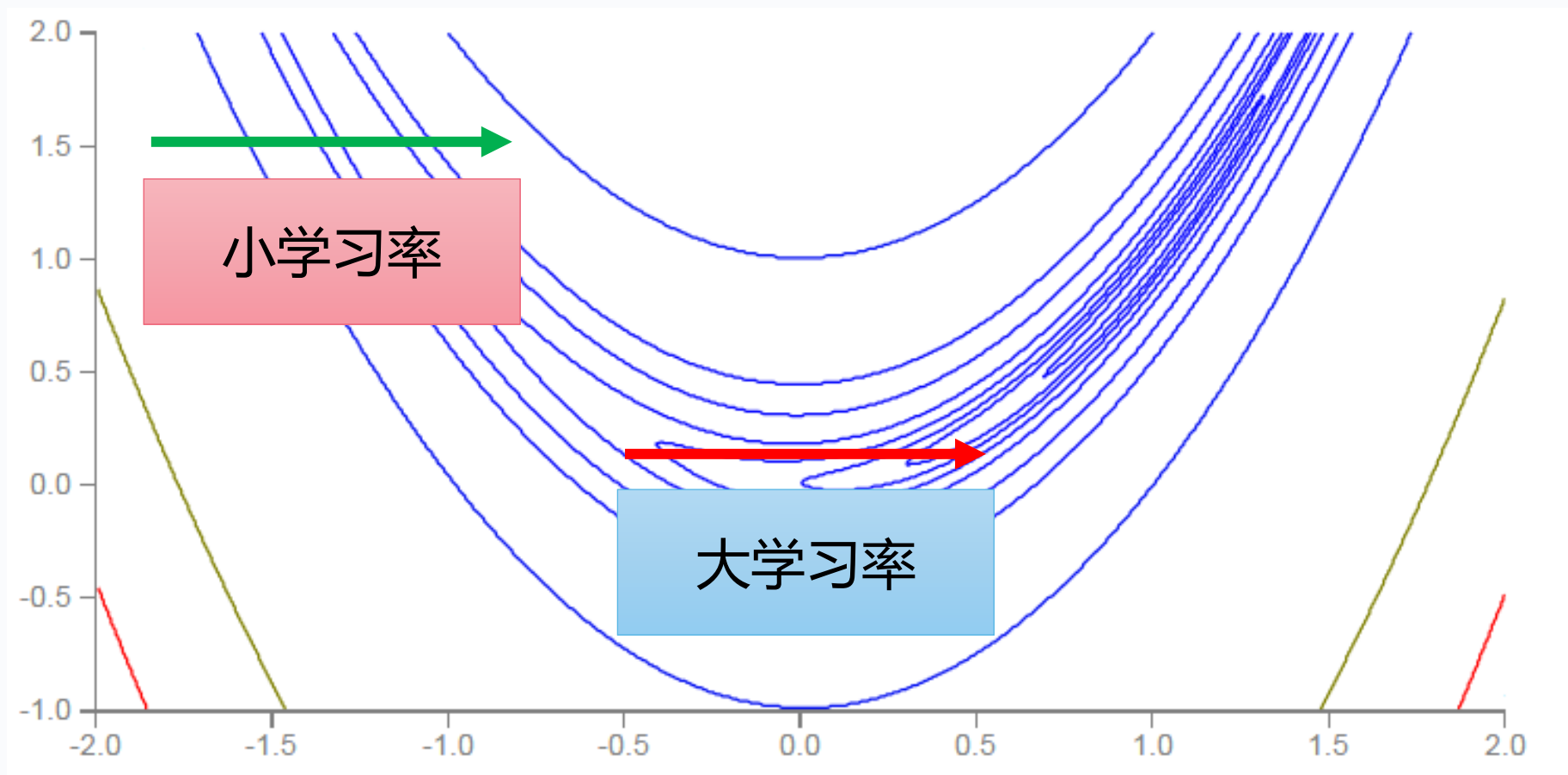




北京外国语大学
BEIJING FOREIGN STUDIES UNIVERSITY

示例代码

同一个参数的Gradient大小不会一成不变



同一个参数的Gradient大小不会一成不变

$$g^0 = \begin{bmatrix} 500 \\ 0.4 \end{bmatrix} \quad g^1 = \begin{bmatrix} 432 \\ -0.3 \end{bmatrix} \quad \dots \quad g^{t-1} = \begin{bmatrix} -0.1 \\ 229 \end{bmatrix} \quad g^t = \begin{bmatrix} 0.1 \\ 100 \end{bmatrix}$$

变大 变大



Adagrad: 全部平方相加

RMSProp: 最近算出的 gradient 赋予较大的影响

同一个参数的Gradient大小不会一成不变

RMSProp

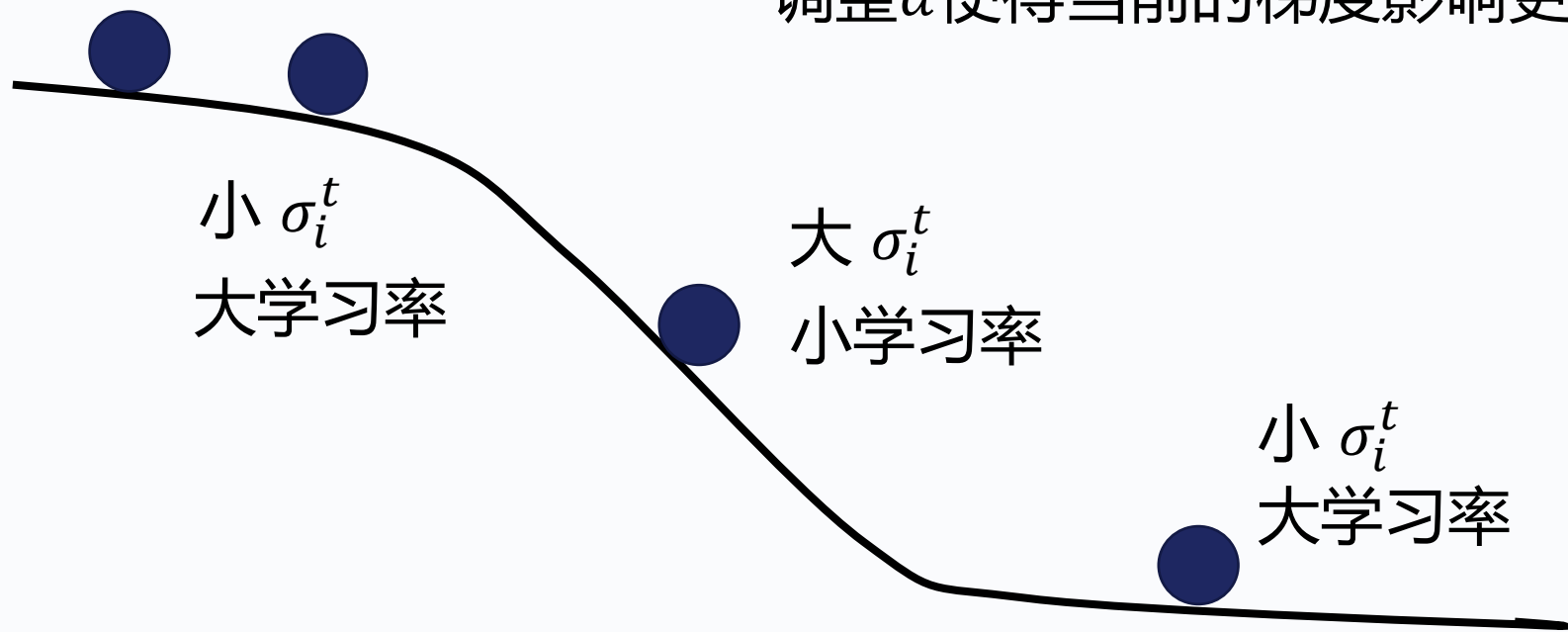
$$\theta_i^{t+1} \leftarrow \theta_i^t - \frac{\eta}{\sigma_i^t} g_i^t$$

$$\sigma_i^t = \sqrt{\alpha (\sigma_i^{t-1})^2 + (1 - \alpha) (g_i^t)^2}$$

$g_i^1 \ g_i^2 \ \dots \ g_i^{t-1}$
 $\vdots$

$0 < \alpha < 1$

调整 α 使得当前的梯度影响更大



同一个参数的Gradient大小不会一成不变

RMSProp: 改进的Adagrad

$$\theta_i^{t+1} \leftarrow \theta_i^t - \frac{\eta}{\sigma_i^t} g_i^t$$

$$\theta_i^1 \leftarrow \theta_i^0 - \frac{\eta}{\sigma_i^0} g_i^0$$

$$\theta_i^2 \leftarrow \theta_i^1 - \frac{\eta}{\sigma_i^1} g_i^1$$

$$\theta_i^3 \leftarrow \theta_i^2 - \frac{\eta}{\sigma_i^2} g_i^2$$

⋮

$$\theta_i^{t+1} \leftarrow \theta_i^t - \frac{\eta}{\sigma_i^t} g_i^t$$

$$\sigma_i^0 = \sqrt{(g_i^0)^2} \quad 0 < \alpha < 1$$

$$\sigma_i^1 = \sqrt{\alpha(\sigma_i^0)^2 + (1 - \alpha)(g_i^1)^2}$$

$$\sigma_i^2 = \sqrt{\alpha(\sigma_i^1)^2 + (1 - \alpha)(g_i^2)^2}$$

$$\sigma_i^t = \sqrt{\alpha(\sigma_i^{t-1})^2 + (1 - \alpha)(g_i^t)^2}$$

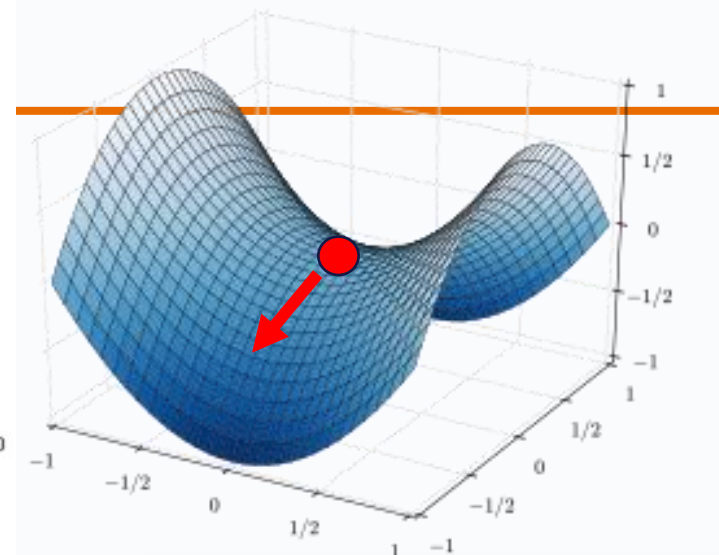
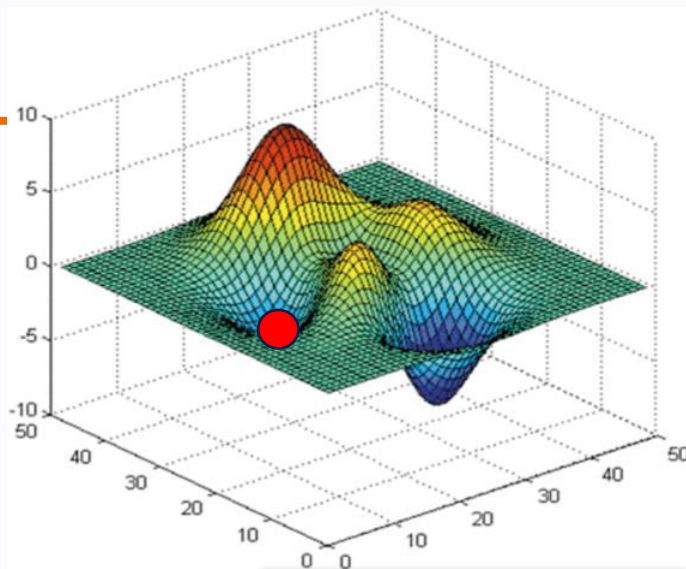
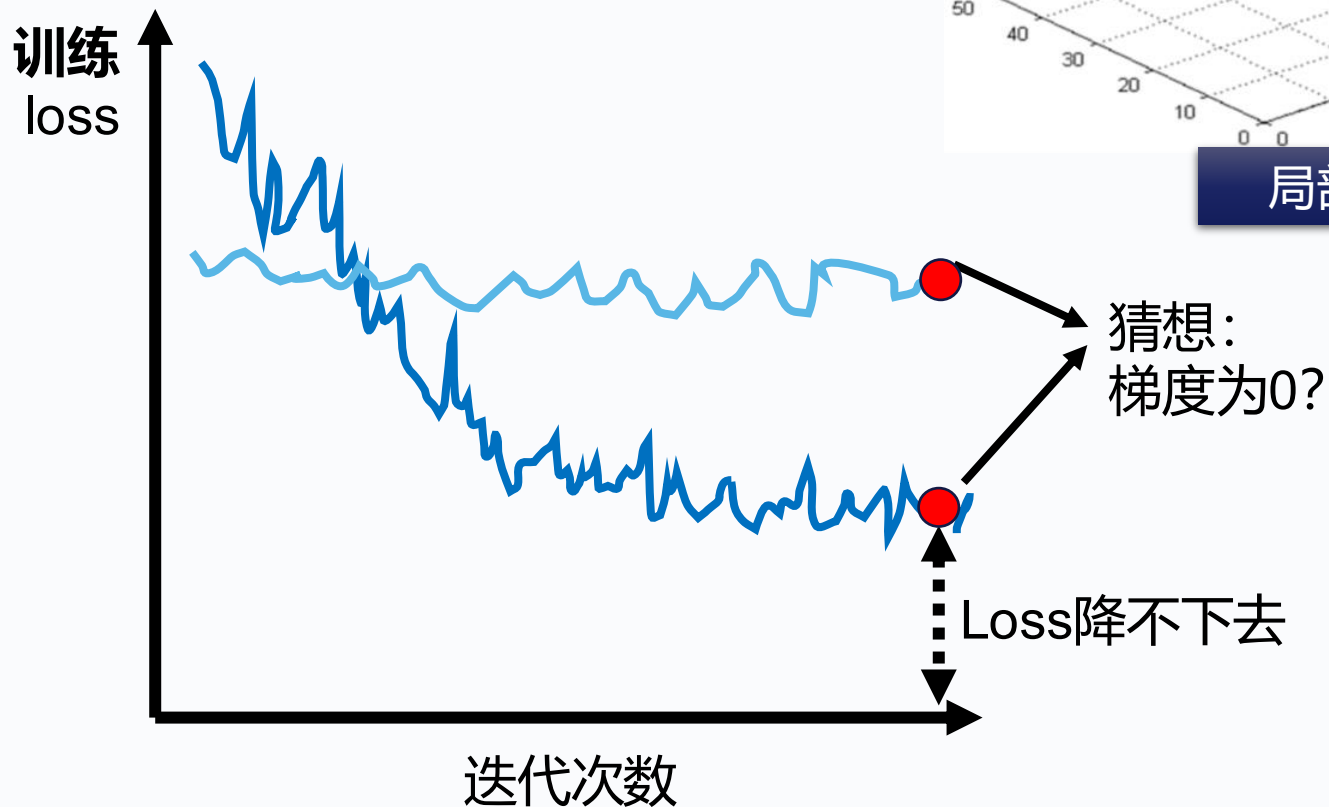


北京外国语大学
BEIJING FOREIGN STUDIES UNIVERSITY

示例代码

优化问题

为什么训练loss降不下去？



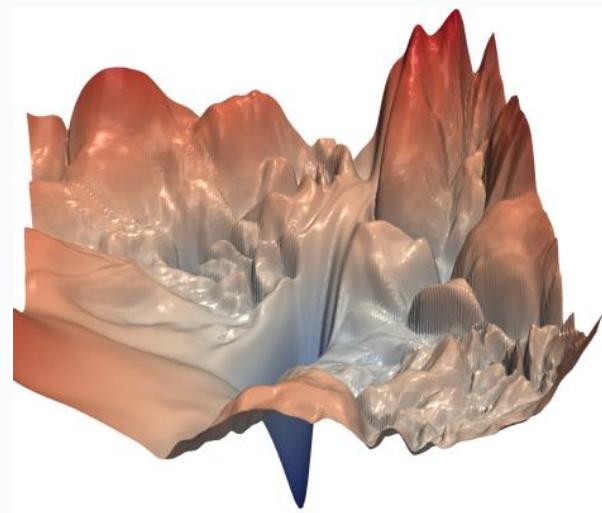
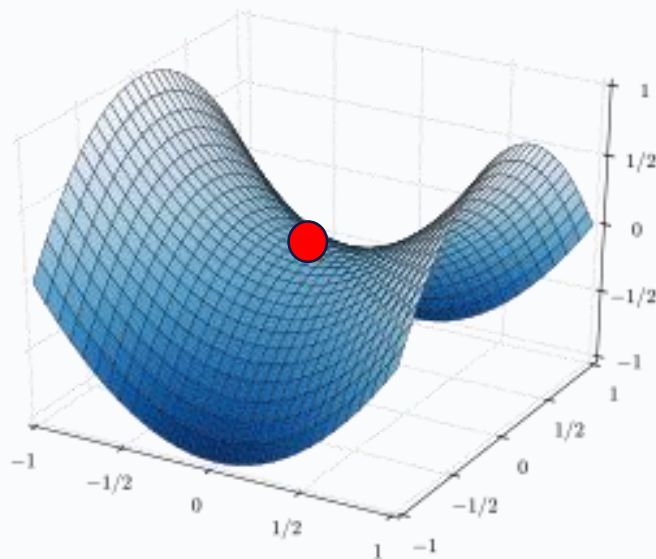
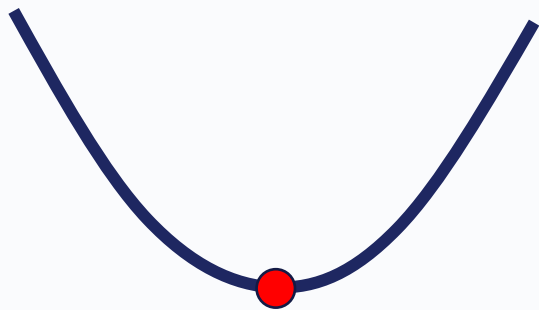
临界点
(critical point)

优化问题

鞍点其实一点也不可怕！在梯度方向上引入随机性，可以有效地逃离鞍点。

梯度为0时，碰到的是鞍点？还是局部极值点？

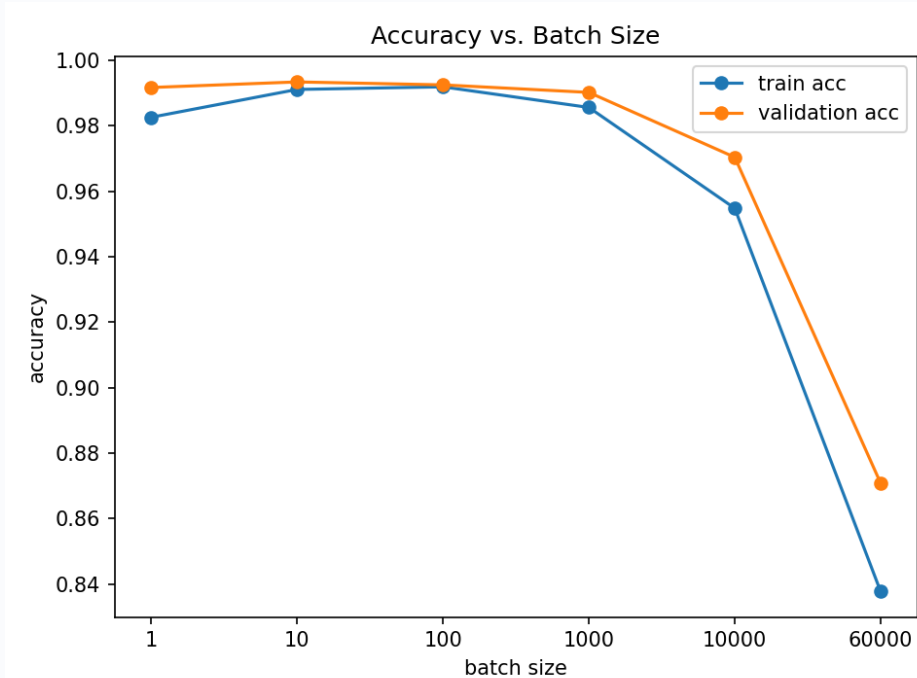
当参数量足够大时，几乎没有局部极小值点。



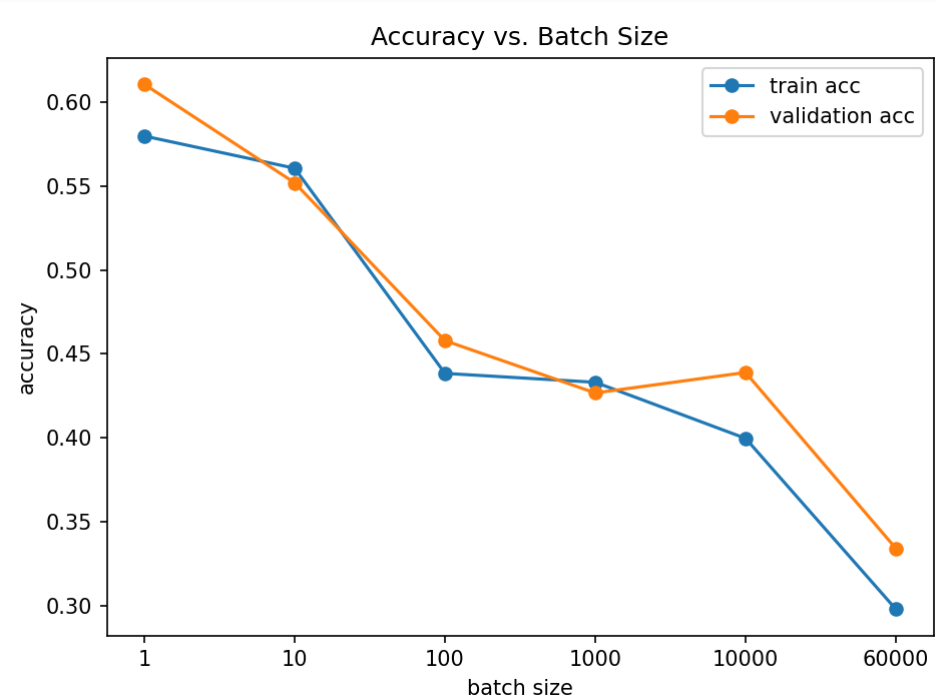
来点随机也挺好

例子：

MNIST



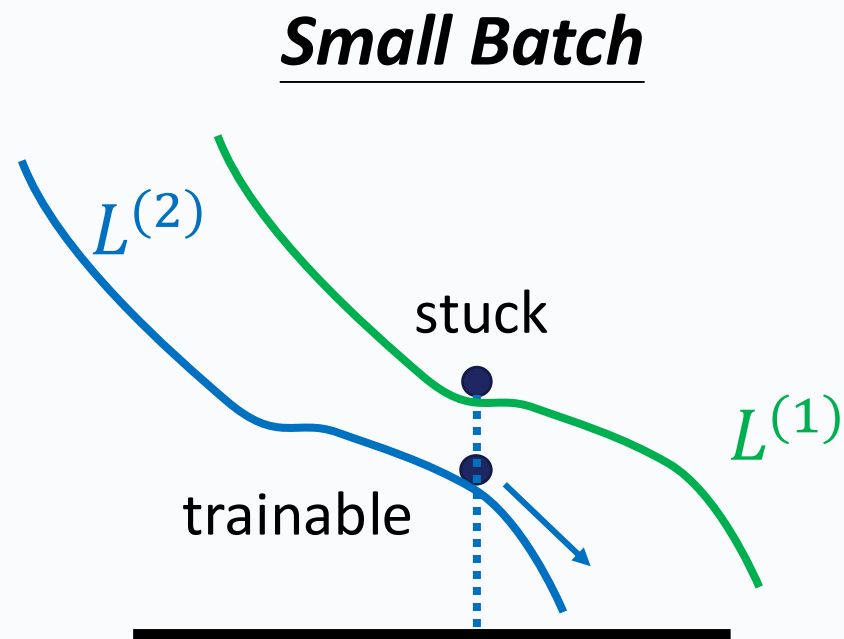
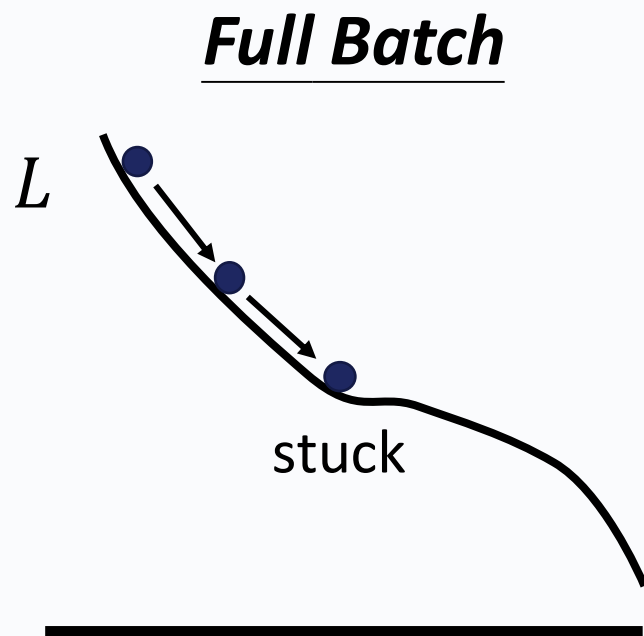
CIFAR-10



小的Batch Size似乎会带来更好的优化结果。为什么？

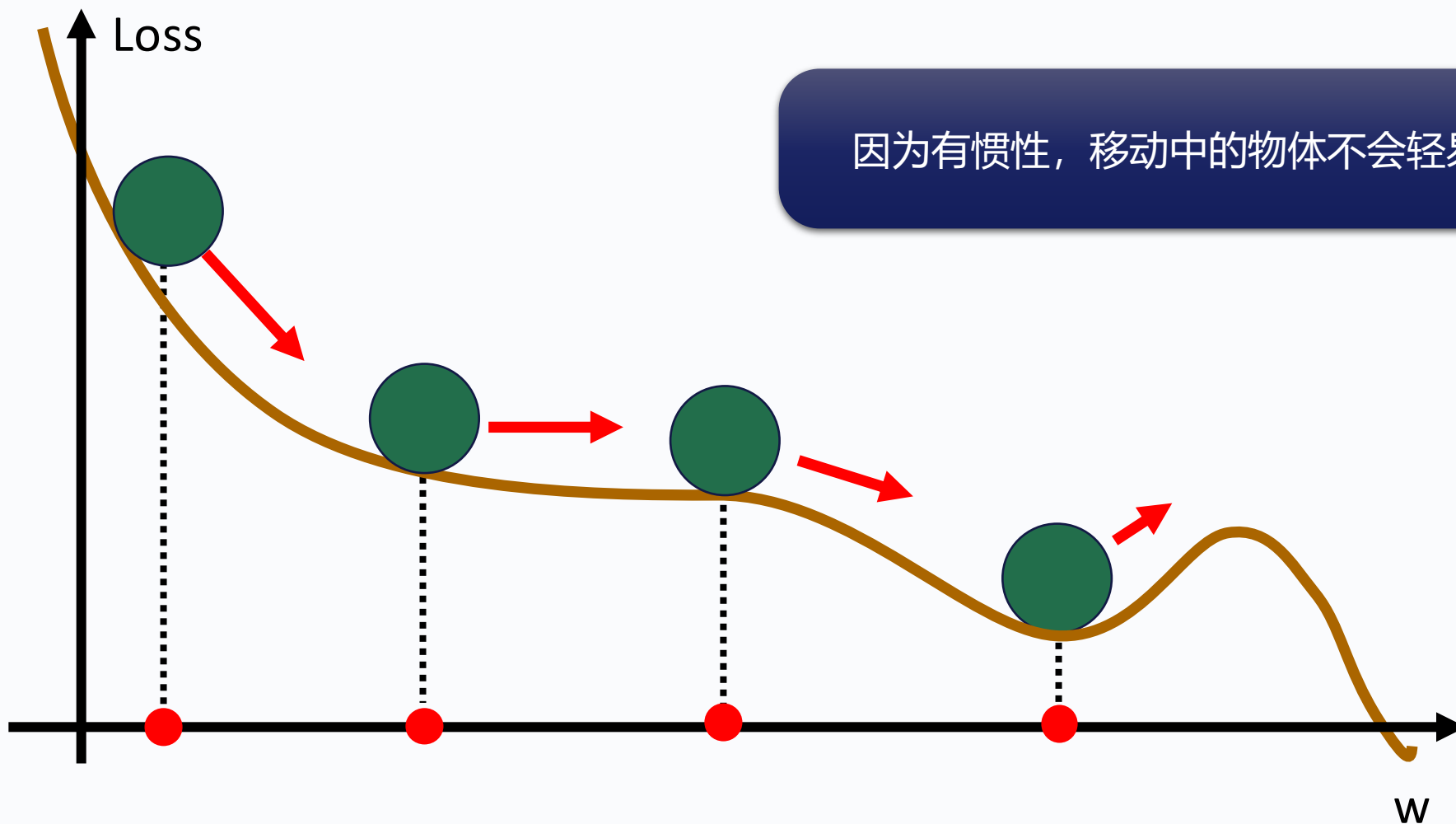
来点随机也挺好

小 Batch 带来的噪声也许有益于优化



物理世界小球的运动

动量



帶有動量的梯度下降

- Compute gradient $\mathbf{g}^0 = \nabla L(\boldsymbol{\theta}^0)$

For each dimension i : $m_i^0 = g_i^0$

$$\theta_i^1 \leftarrow \theta_i^0 - \eta m_i^0$$

- Compute gradient $\mathbf{g}^1 = \nabla L(\boldsymbol{\theta}^1)$

For each dimension i : $m_i^1 = g_i^0 + g_i^1$

$$\theta_i^2 \leftarrow \theta_i^1 - \eta m_i^1$$

- Compute gradient $\mathbf{g}^2 = \nabla L(\boldsymbol{\theta}^2)$

⋮

For each dimension i : $m_i^2 = g_i^0 + g_i^1 + g_i^2$

$$\theta_i^3 \leftarrow \theta_i^2 - \eta m_i^2$$

- Compute gradient $\mathbf{g}^t = \nabla L(\boldsymbol{\theta}^t)$

For each dimension i : $m_i^t = g_i^0 + g_i^1 + g_i^2 + \cdots + g_i^t$

$$\theta_i^{t+1} \leftarrow \theta_i^t - \eta m_i^t$$

帶有動量的梯度下降

- Compute gradient $\mathbf{g}^0 = \nabla L(\boldsymbol{\theta}^0)$

For each dimension i : $\mathbf{m}_i^0 = \mathbf{g}_i^0$

$$\theta_i^1 \leftarrow \theta_i^0 - \eta \mathbf{m}_i^0$$

- Compute gradient $\mathbf{g}^1 = \nabla L(\boldsymbol{\theta}^1)$ $0 < \beta < 1$

For each dimension i : $\mathbf{m}_i^1 = \beta \mathbf{m}_i^0 + (1 - \beta) \mathbf{g}_i^1$

$$\theta_i^2 \leftarrow \theta_i^1 - \eta \mathbf{m}_i^1$$

- Compute gradient $\mathbf{g}^2 = \nabla L(\boldsymbol{\theta}^2)$

⋮ For each dimension i : $\mathbf{m}_i^2 = \beta \mathbf{m}_i^1 + (1 - \beta) \mathbf{g}_i^2$

$$\theta_i^3 \leftarrow \theta_i^2 - \eta \mathbf{m}_i^2$$

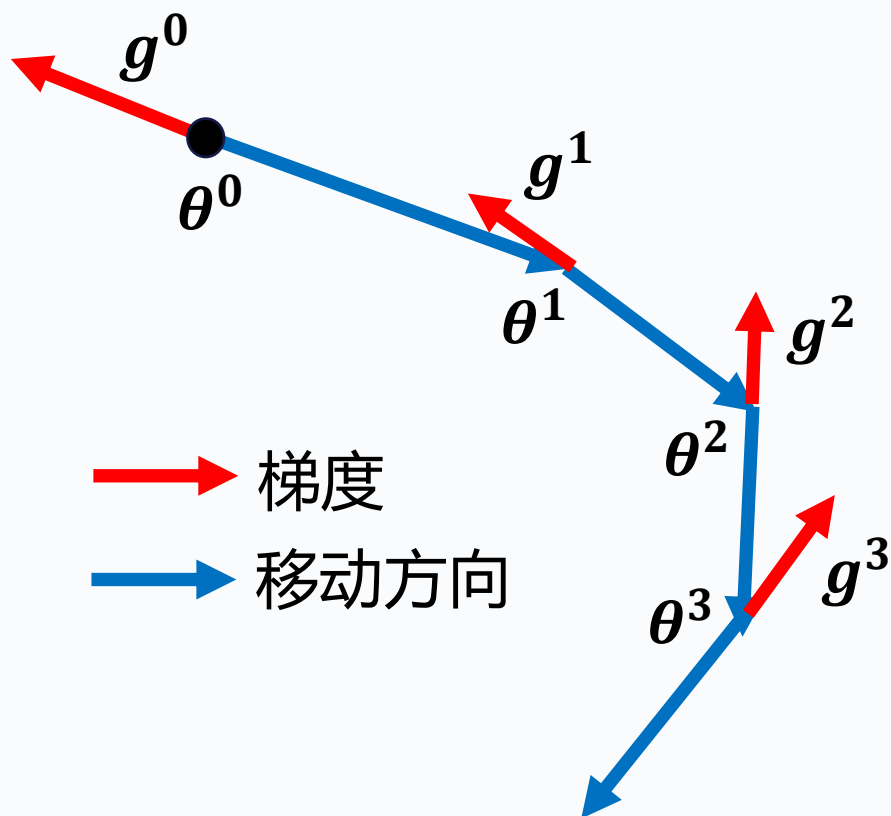
- Compute gradient $\mathbf{g}^t = \nabla L(\boldsymbol{\theta}^t)$

For each dimension i : $\mathbf{m}_i^t = \beta \mathbf{m}_i^{t-1} + (1 - \beta) \mathbf{g}_i^t$

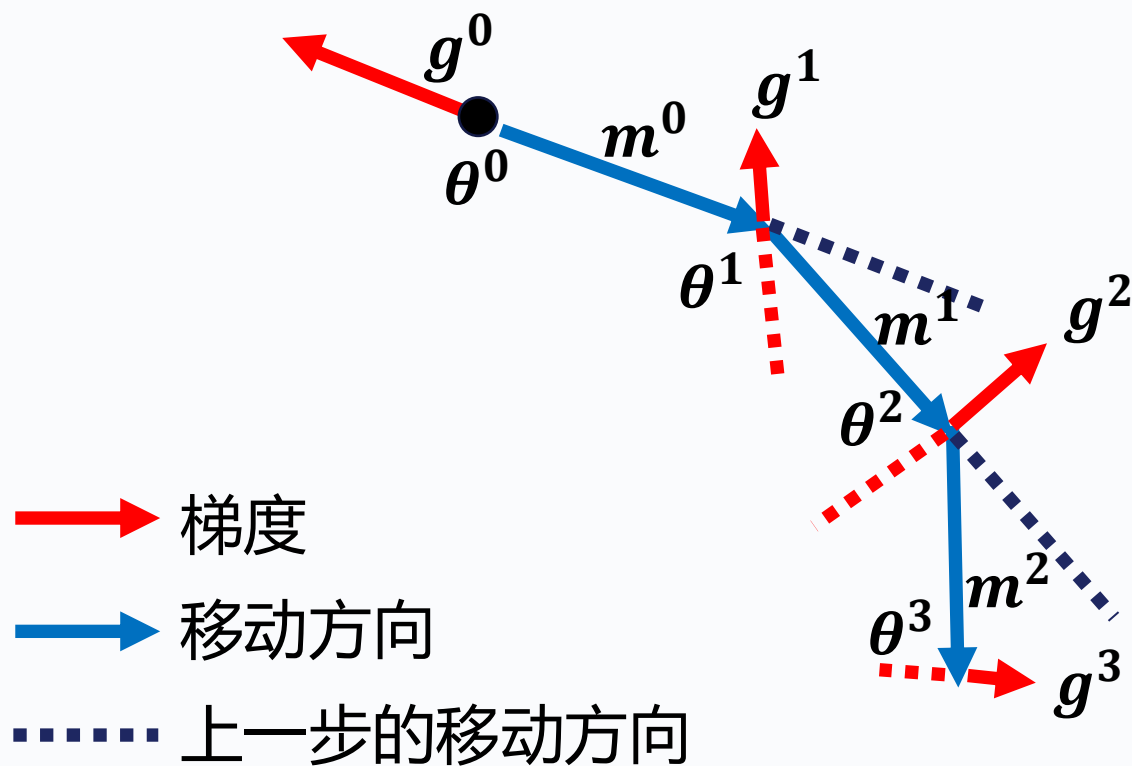
$$\theta_i^{t+1} \leftarrow \theta_i^t - \eta \mathbf{m}_i^t$$

梯度下降 + 动量

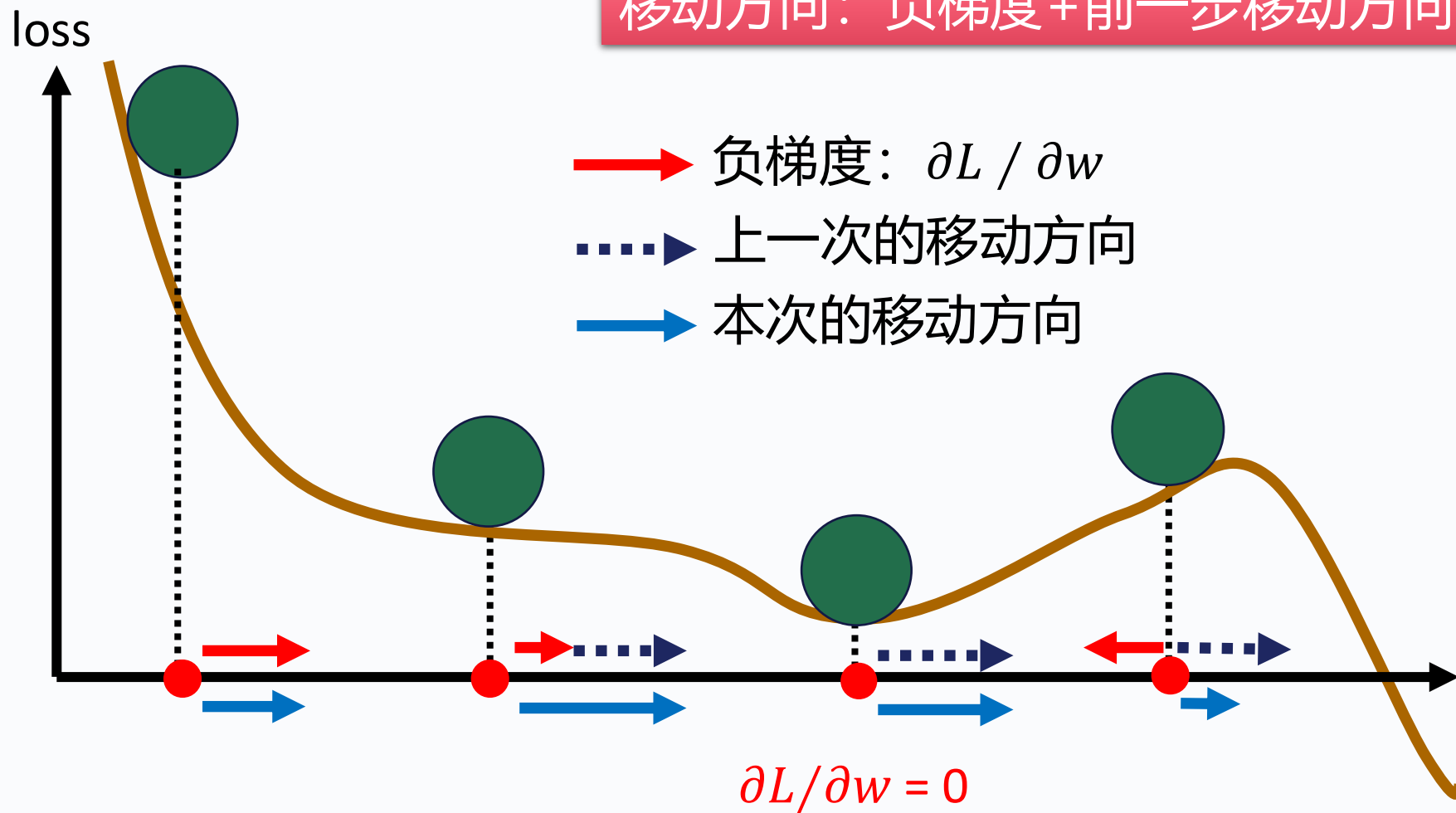
原始的梯度下降



不仅只看当前，还要考虑前一步的移动方向



梯度下降 + 动量



Adam: RMSProp + Momentum

- Compute gradient $\mathbf{g}^t = \nabla L(\boldsymbol{\theta}^t)$

Momentum

For each dimension i : $m_i^t = \beta m_i^{t-1} + (1 - \beta) g_i^t$ $\theta_i^{t+1} \leftarrow \theta_i^t - \eta m_i^t$

RMSProp

For each dimension i : $\sigma_i^t = \sqrt{\alpha (\sigma_i^{t-1})^2 + (1 - \alpha) (g_i^t)^2}$ $\theta_i^{t+1} \leftarrow \theta_i^t - \frac{\eta}{\sigma_i^t} g_i^t$

Adam (2015)

$$\theta_i^{t+1} \leftarrow \theta_i^t - \frac{\eta}{\sigma_i^t} m_i^t$$

为了便于理解，与标准的Adam有细微的差异



北京外国语大学
BEIJING FOREIGN STUDIES UNIVERSITY

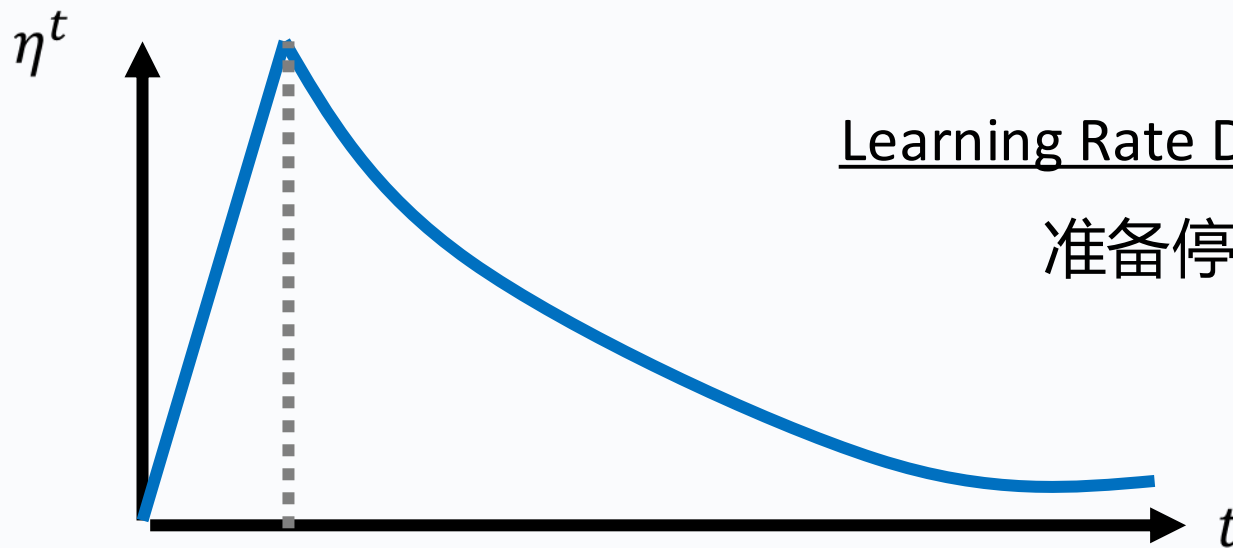
示例代码

Learning Rate Scheduling

$$\theta_i^{t+1} \leftarrow \theta_i^t - \frac{\eta}{\sigma_i^t} m_i^t \quad \longrightarrow \quad \theta_i^{t+1} \leftarrow \theta_i^t - \frac{\eta^t}{\sigma_i^t} m_i^t$$

Warm Up

探索地形



Learning Rate Decay

准备停下来

总结

$$\theta_i^{t+1} \leftarrow \theta_i^t - \frac{\eta^t}{\sigma_i^t} m_i^t$$

Learning rate scheduling

动量: sum of the previous gradients

square of the gradients

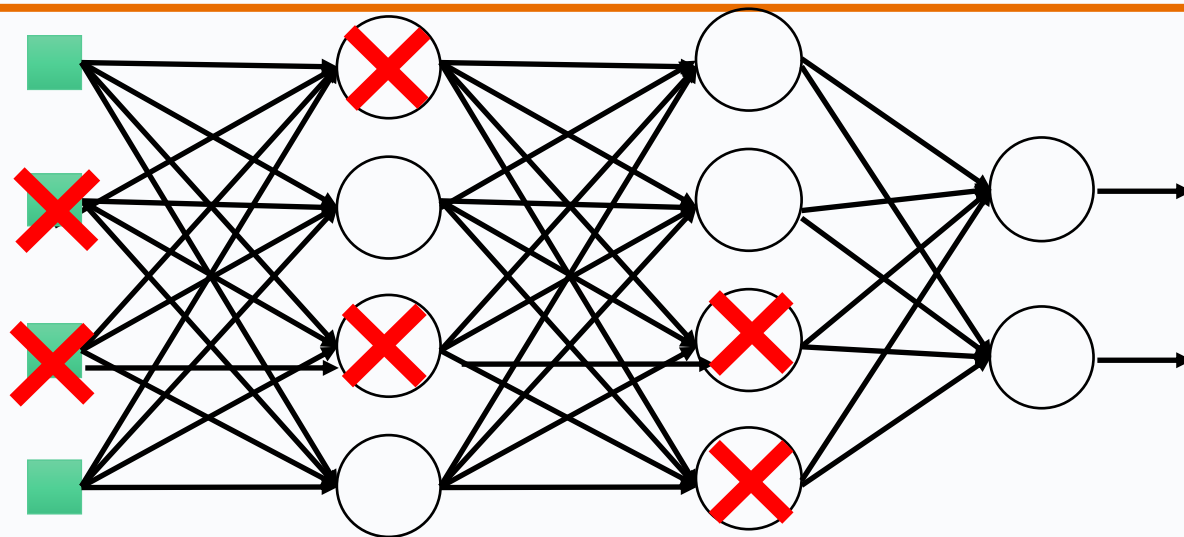
考虑方向

仅考虑数值大小

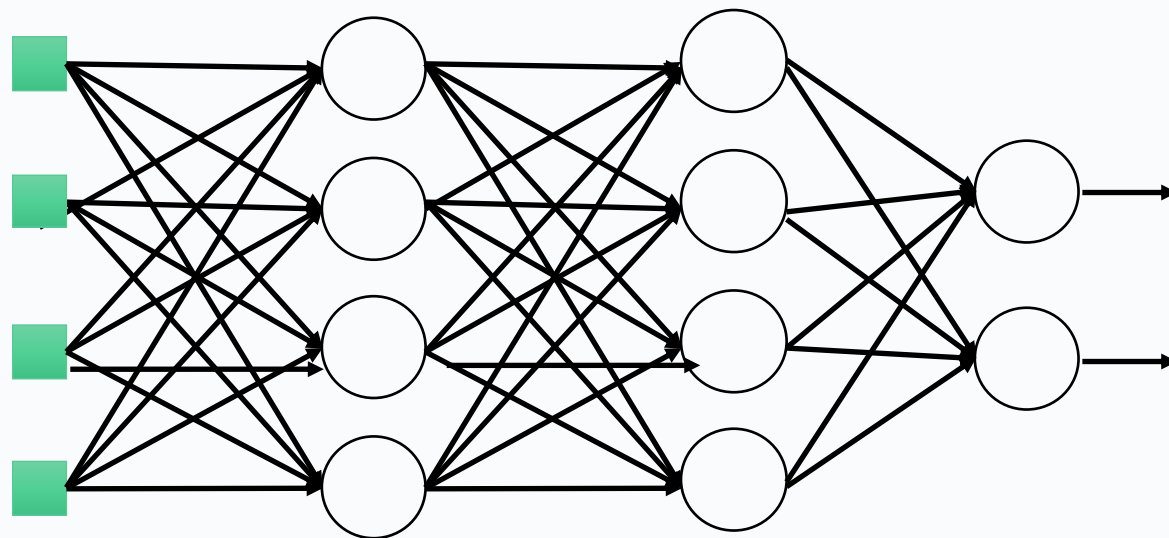
方法名称	改了哪个步骤	带来什么好处
Adagrad, RMSProp, Momentum, Adam, etc.	寻找最好的函数	更好的优化 (不是为了泛化)

Dropout

训练时随机丢掉
一些神经元



验证和测试时
火力全开

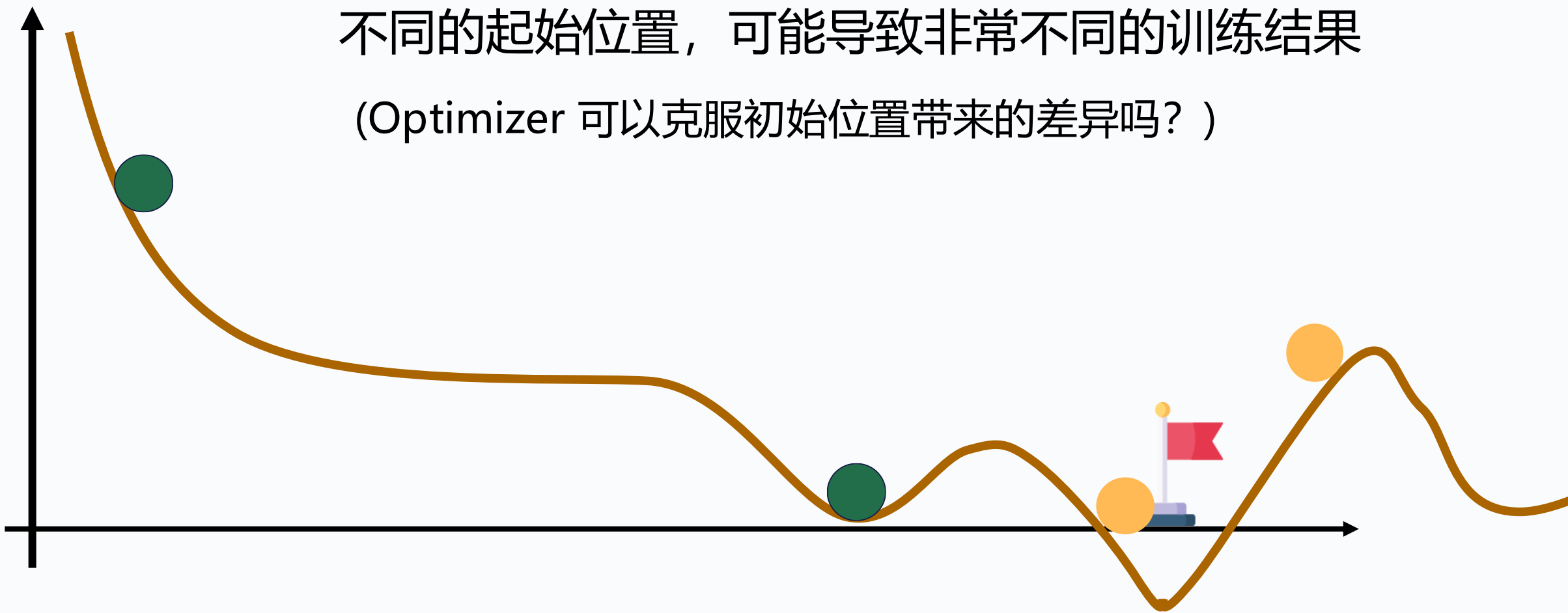


Dropout

方法名称	改了哪个步骤	带来什么好处
Dropout	寻找最好的函数	更好的泛化性能 (更差的优化过程)

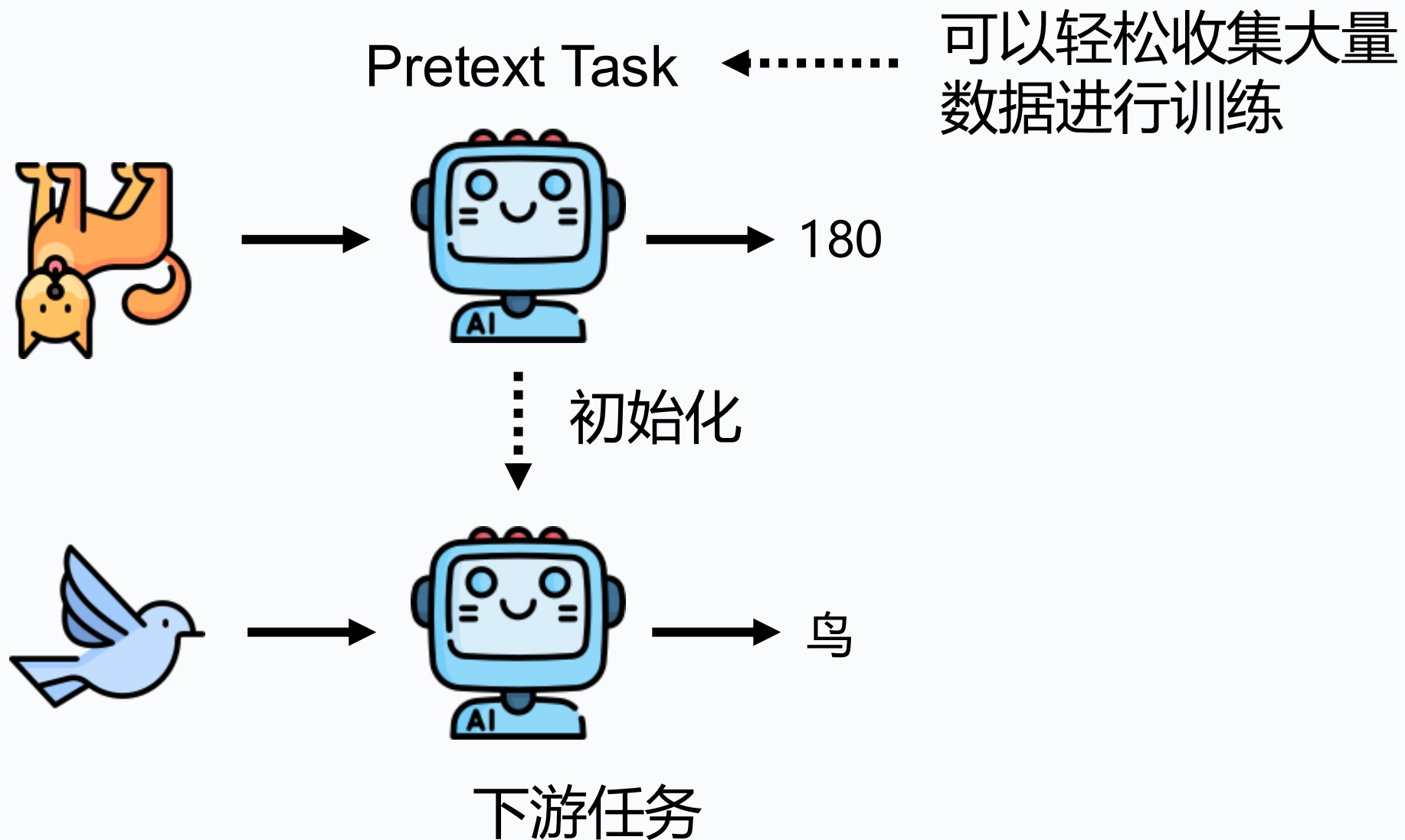
Initialization

不同的起始位置，可能导致非常不同的训练结果
(Optimizer 可以克服初始位置带来的差异吗？)



Initialization

预训练

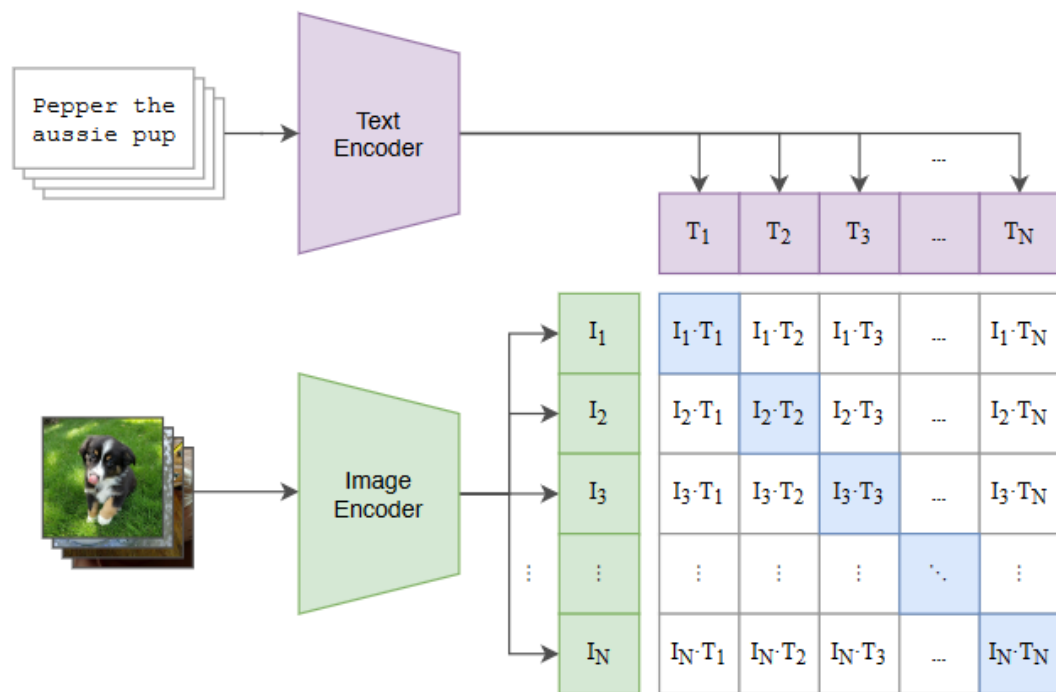


CLIP

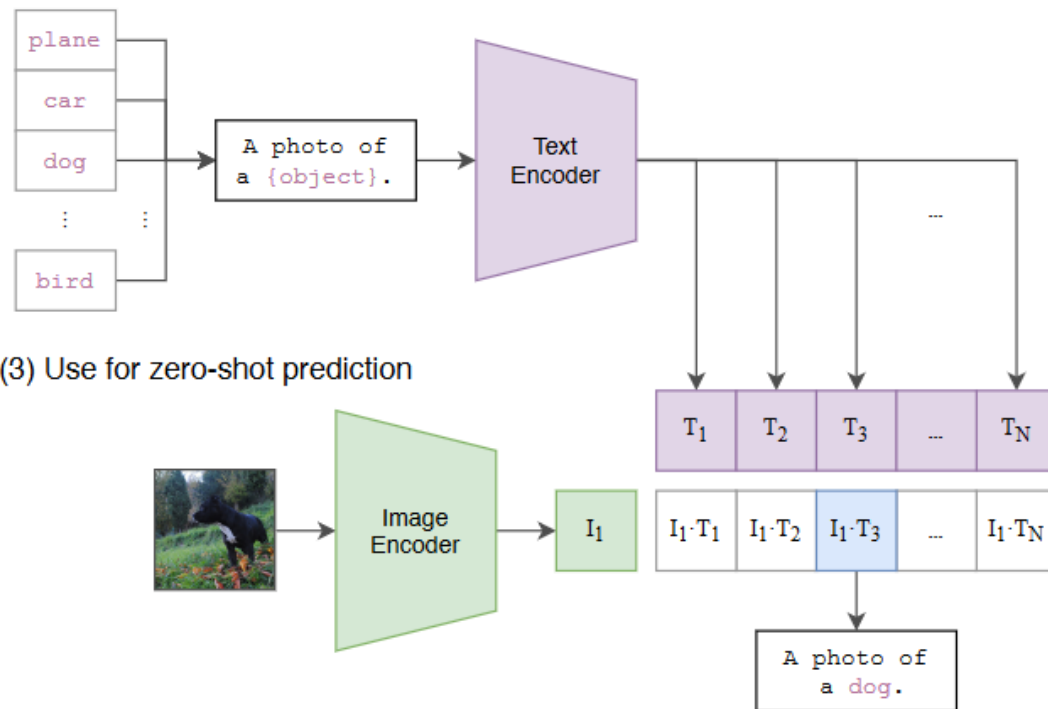
<https://arxiv.org/abs/2103.00020>

Contrastive Language-Image Pre-training

(1) Contrastive pre-training

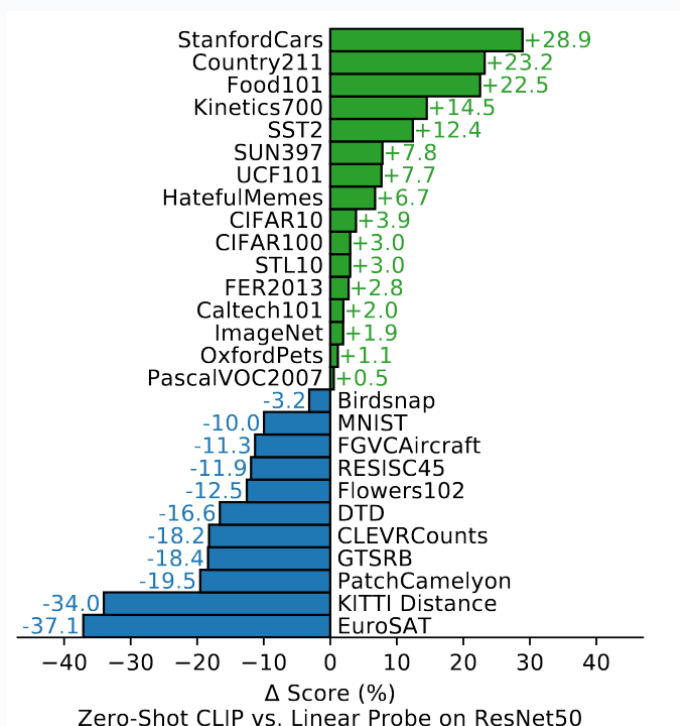


(2) Create dataset classifier from label text



CLIP

<https://arxiv.org/abs/2103.00020>



Zero-shot CLIP is competitive with a fully supervised baseline.

大数据+大模型=效果炸裂

Money is all you need.. (误)



钞能力

DINO

<https://arxiv.org/abs/2104.14294>

Initialization

Pre-train 对于 Optimization 和 Generalization 都有帮助

方法名称	改了哪个步骤	带来什么好处
Initialization (e.g., pre-train)	寻找最好的函数	Better Optimization, Better Generalization